

Lecture 9: Verification of Neural Networks

CS 580B1: Trustworthy Machine Learning

Fall 2024

Acknowledgments

Course slides derived from material created by Rajeev Alur and Osbert Bastani at UPenn ([link](#) to their course)

Verification of Machine Learning Programs by Guy Katz at Hebrew University of Jerusalem

[Trustworthy AI Systems](#) at UIUC by Gagandeep Singh and Madhu Parthasarathy

Other relevant courses on Trustworthy Machine Learning:

[CS 329T](#) at Stanford

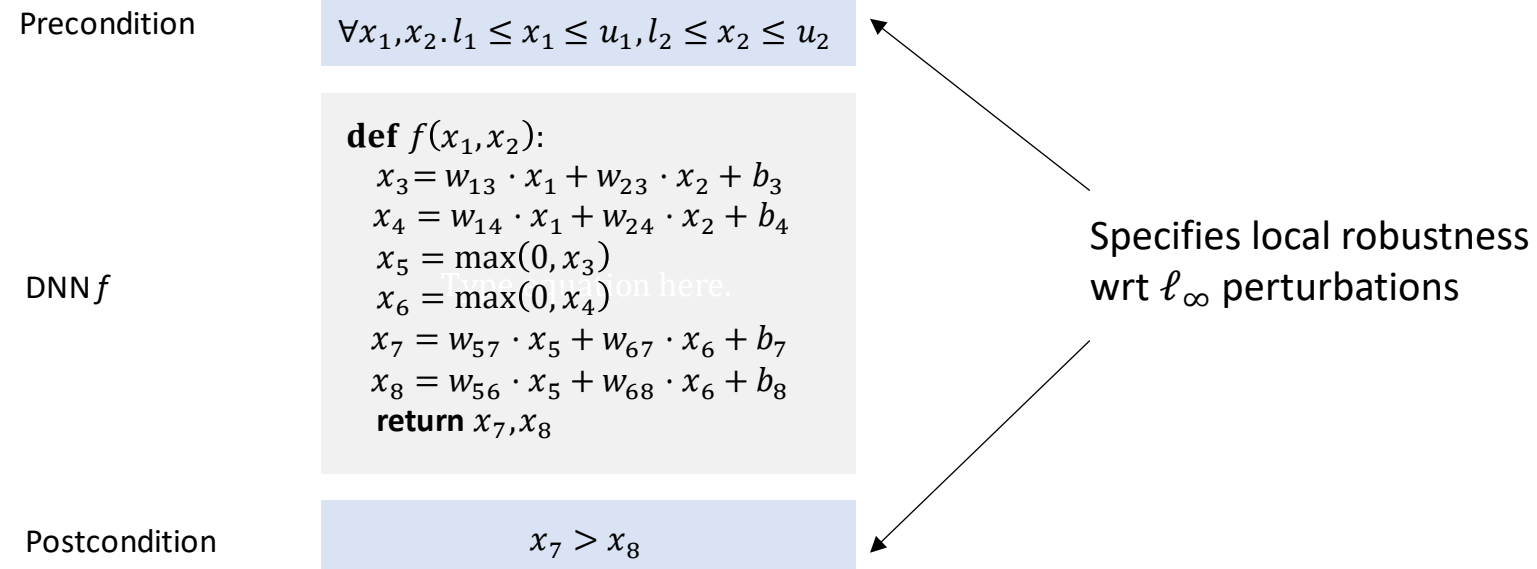
[CS 521](#) at UIUC

[ECE1784H/CSC2559H](#) at U Toronto

Course logistics

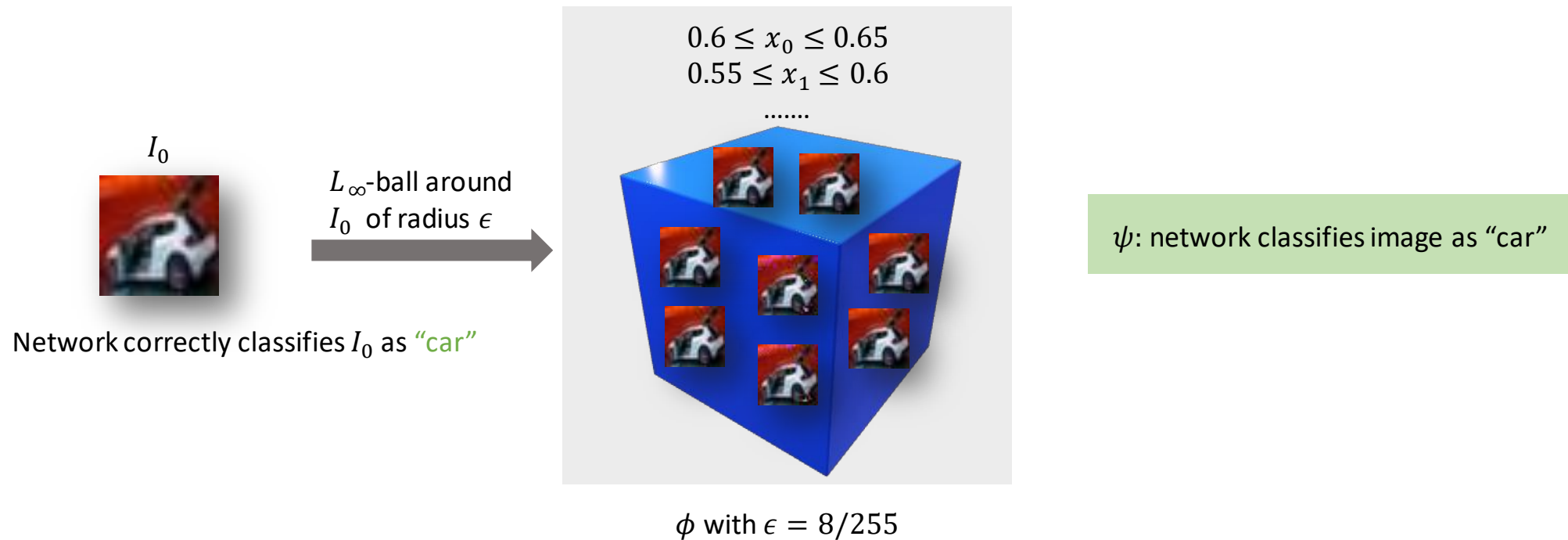
- Reviews:
 - Review due tomorrow at 11:59 PM
 - Paper to review: [Formal Security Analysis of Neural Networks using Symbolic Intervals](#) by Wang et al.
 - Mention names of those who you discussed the paper with in your review
- Projects:
 - Form teams!
 - Proposal format will be out soon
- Can submit anonymous feedback about course during the semester

Recap: Formal specifications for DNNs

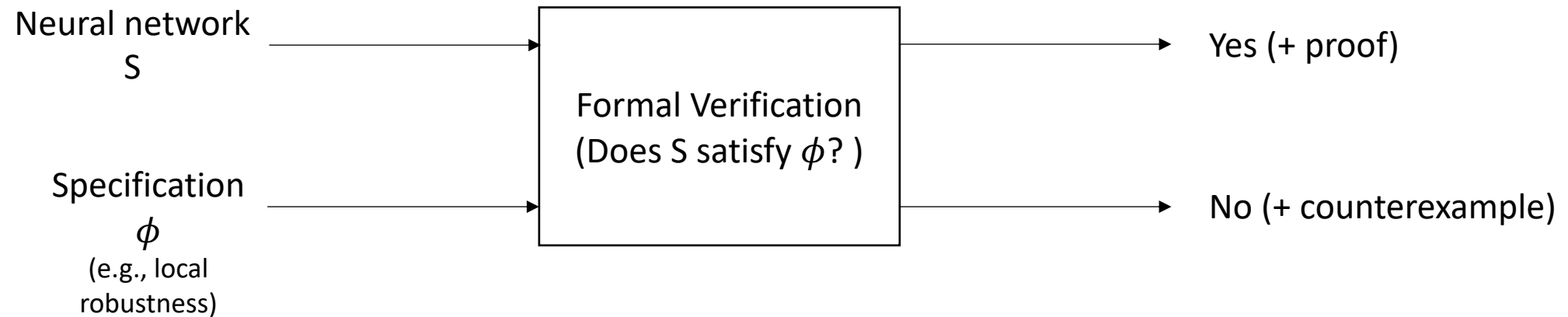


Either prove that the network output satisfies the postcondition for all inputs in the pre-condition or find a counterexample

Recap: Formal specifications for DNNs--Local robustness



Recap: Formal verification of DNNs



Recap: Techniques for Formal verification of DNNs

Incomplete	Abstract interpretation: Box , Zonotope, DeepPoly
Complete	Mixed Integer Linear Programming (MILP) SMT solvers (Reluplex)

Active area of research with annual competition: VNNComp
Current winner: alpha-beta crown

DNN verification as constraint solving

Precondition

$$\forall x_1, x_2. l_1 \leq x_1 \leq u_1, l_2 \leq x_2 \leq u_2$$

DNN f

```
def f(x1, x2):  
    x3 = w13 · x1 + w23 · x2 + b3  
    x4 = w14 · x1 + w24 · x2 + b4  
    x5 = max(0, x3)  
    x6 = max(0, x4)  
    x7 = w57 · x5 + w67 · x6 + b7  
    x8 = w56 · x5 + w68 · x6 + b8  
    return x7, x8
```

Postcondition

$$x_7 > x_8$$

- Set of variables real-valued variables V
- Atomic predicates:
 - Linear inequalities of the form $\sum_{v_i \in V} a_i v_i \leq c_i$
 - ReLU equation of the form $v_i = \text{ReLU}(v_j)$
- DNN represented by a conjunction of atomic predicates $\psi_N = \psi_1 \wedge \dots \wedge \psi_t$
- Precondition ψ_{pre} and postcondition ψ_{post}
- Check satisfiability of $\psi_{pre} \wedge \psi_N \wedge \neg \psi_{post}$

Reluplex

- Proposed in paper “Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks” by Katz et al. in CAV’17
- SMT-solver for quantifier-free linear real arithmetic + ReLUs
- Based on the *Simplex* method for checking satisfiability of conjunctions of linear real arithmetic constraints
 - *Simplex* + ReLUs = Reluplex
 - Applicable to other piece-wise linear functions
- *Simplex* algorithm developed shortly after WW2 by George Dantzig
 - Solves linear programs, i.e., constrained optimization problems with linear objective function and linear equations as constraints
 - Divided into two phases:
 - Find a feasible solution (Our focus)
 - Optimize with respect to objective function

The *Simplex* algorithm

- Solves satisfiability of conjunction of linear inequalities

$$\varphi = v_1 + v_2 \leq -5 \wedge v_1 - v_2 \geq 3$$

Add one new variable for each inequality

- Step 1: Construct an initial **configuration**

$u: V \rightarrow \mathbb{R} \cup \{\infty\}$
 $l: V \rightarrow \mathbb{R} \cup \{-\infty\}$
Upper and lower bounds on variable

$$V = \{v_1, v_2, v_3, v_4\}$$

Set of **basic** variable $B \subseteq V$
Initially it is all new variables

- $B = \{v_3, v_4\}$
- $T: v_3 = v_1 + v_2$ and $v_4 = v_1 - v_2$
- $u(v_3) = -5$ and $l(v_4) = 3$
- $\alpha(v_i) = 0$ for all $v_i \in V$

Tableau T contains one equation per basic variable; **RHS only has non-basic variables**

Valuation satisfies T

Goal of the algorithm: Update current values of non-basic variables to meet all lower/upper bounds

The *Simplex* algorithm

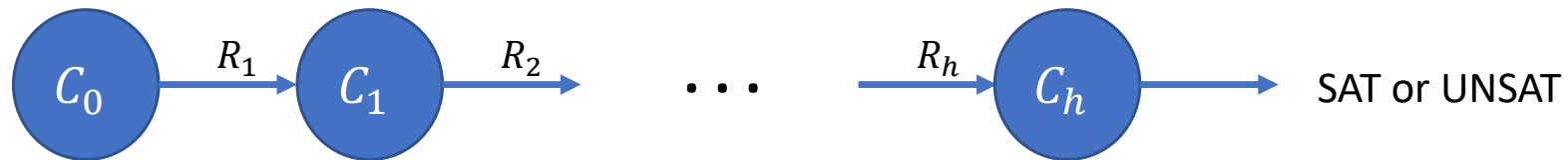
Simplex provides rules of the following kind for modifying a configuration $\mathcal{C} = (V, B, T, u, l, \alpha)$

$$\frac{\text{Some condition on } \mathcal{C}}{\text{SAT}}$$

$$\frac{\text{Some condition on } \mathcal{C}}{\text{UNSAT}}$$

$$\frac{\text{Some condition on } \mathcal{C}}{\text{Modified configuration } \mathcal{C}'}$$

Apply rules until a SAT or UNSAT is derived



The *Simplex* algorithm

- Iterative algorithm
- Always maintains a variable assignment α
- Assignment always satisfies equations
 - But may violate bounds
- In every iteration, attempt to reduce the overall infeasibility

The *Simplex* algorithm

- Variables partitioned into **basic** (***B***) and **non-basic** variables
 - Non-basics are “free”
 - Basics are “bounded”
- Non-basic assignment dictates basic assignment
 - Ensures the equations are maintained
- In every iteration, we can perform
 - an **update**: change the assignment of a non-basic variable and any affected basics
 - a **pivot**: switch a basic and non-basic variable

Slack variables

Coefficient of v_j in RHS defining v_i

$$\text{slack}^+(v_i) = \{v_j \notin B \mid (T_{i,j} > 0 \wedge \alpha(v_j) < u(v_j)) \vee (T_{i,j} < 0 \wedge \alpha(v_j) > l(v_j))\}$$

$$\text{slack}^-(v_i) = \{v_j \notin B \mid (T_{i,j} < 0 \wedge \alpha(v_j) < u(v_j)) \vee (T_{i,j} > 0 \wedge \alpha(v_j) > l(v_j))\}$$

$\text{slack}^+(v_i)$: Variables on RHS defining v_i whose values can be changed to **increase** the value of v_i
e.g. variable v_j has positive coefficient and its current value is less than its upper bound

$\text{slack}^-(v_i)$: Variables on RHS defining v_i whose values can be changed to **decrease** the value of v_i

Slack variables

$$\text{slack}^+(v_i) = \{v_j \notin B \mid (T_{i,j} > 0 \wedge \alpha(v_j) < u(v_j)) \vee (T_{i,j} < 0 \wedge \alpha(v_j) > l(v_j))\}$$

$$\text{slack}^-(v_i) = \{v_j \notin B \mid (T_{i,j} < 0 \wedge \alpha(v_j) < u(v_j)) \vee (T_{i,j} > 0 \wedge \alpha(v_j) > l(v_j))\}$$

$$V = \{v_1, v_2, v_3, v_4\}$$

- $B = \{v_3, v_4\}$
- $T : v_3 = v_1 + v_2$ and $v_4 = v_1 - v_2$
- $u(v_3) = -5$ and $l(v_4) = 3$
- $\alpha(v_i) = 0$ for all $v_i \in V$

$$\text{slack}^+(v_3) = \{v_1, v_2\}$$

If we have $u(v_1) = 0$ then

$$\text{slack}^+(v_3) = \{v_1, v_2\}$$

Simplex rule: Successful termination

$$\frac{\forall v_i \in V, \quad l(v_i) \leq \alpha(v_i) \leq u(v_i)}{\text{SAT}} \quad \text{Success}$$

Current valuation meets all lower/upper bounds: satisfying assignment found

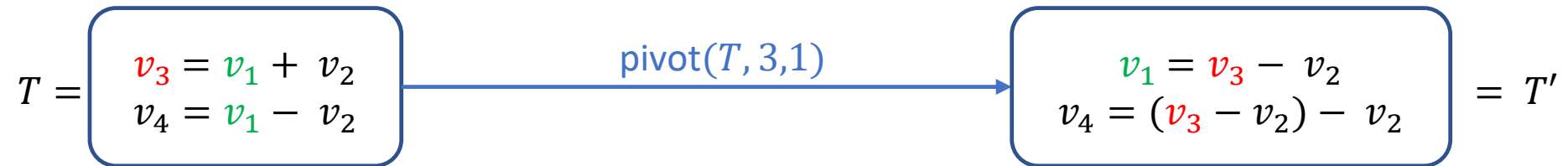
Simplex rule: Pivot

$$\frac{v_i \in B, \quad \alpha(v_i) < l(v_i), \quad v_e \in \mathbf{slack}^+(v_i)}{T := \mathbf{pivot}(T, v_i, v_e), \quad B := B \cup \{v_e\} \setminus \{v_i\}} \quad \mathbf{Pivot}_1$$
$$\frac{v_i \in B, \quad \alpha(v_i) > u(v_i), \quad v_e \in \mathbf{slack}^-(v_i)}{T := \mathbf{pivot}(T, v_i, v_e), \quad B := B \cup \{v_e\} \setminus \{v_i\}} \quad \mathbf{Pivot}_2$$

If: A basic variable's value needs to be increased/decreased to meet lower/upper bound
and there is a possible variable on RHS whose value be changed for this purpose
Then make it non-basic by swapping their roles using pivot

Pivot example

Pivot: Allows replacing basic variable with a non-basic variable



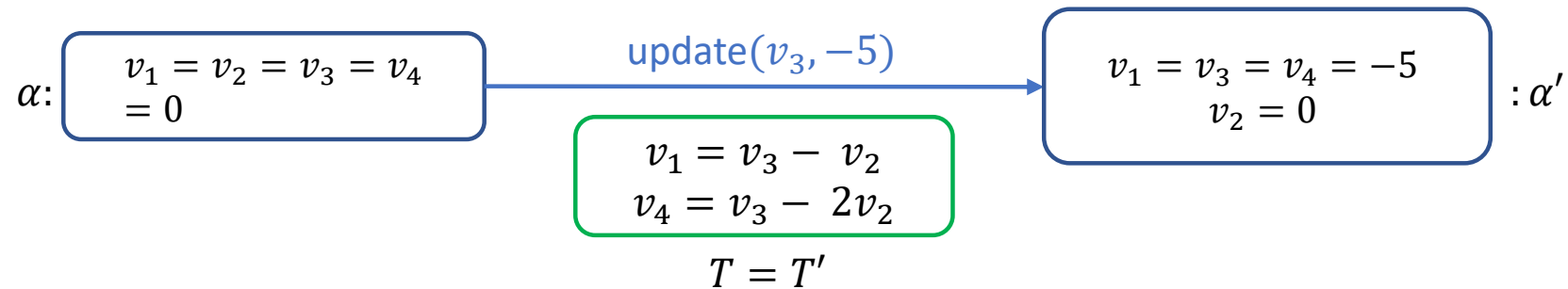
Simplex rule: Update

$$\frac{v_j \notin B, \quad \alpha(v_j) < l(v_j) \vee \alpha(v_j) > u(v_j), \quad l(v_j) \leq \alpha(v_j) + \delta \leq u(v_j)}{\alpha := \text{update}(\alpha, v_j, \delta)} \quad \text{Update}$$

For a non-basic variable, if its current value is less/greater than lower/upper bound then increase/decrease it to meet the bound

Update example

Update: Allows updating value of a non-basic variable



Simplex rule: Unsuccessful termination

$$\frac{v_i \in B, \quad (\alpha(v_i) < l(v_i) \wedge \mathbf{slack}^+(v_i) = \emptyset) \vee (\alpha(v_i) > u(v_i) \wedge \mathbf{slack}^-(v_i) = \emptyset)}{\text{UNSAT}} \quad \text{Failure}$$

There is a basic variable for which current value must be increased/decreased to meet lower/upper bound constraint but no such update of RHS vars is possible

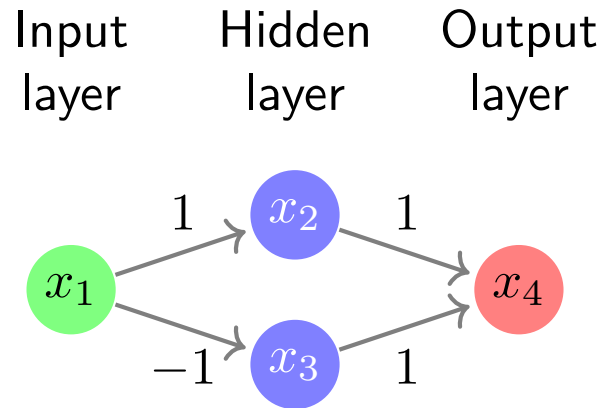
Simplex algorithm in SMT solver

- Starting with initial configuration, keep applying pivot/update rules until the termination condition holds
- Multiple rules may be applicable in a given configuration
 - Derivation tree captures all possible branches of rule applications
- Key engineering details of implementation
 - Which rule to choose in a given configuration
 - Choice of rules affects time to termination but does not require backtracking
 - How to apply rules efficiently (e.g. keeping track of slack variables)

Properties of *Simplex*

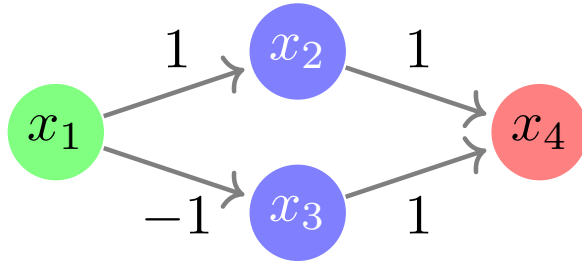
- *Simplex* is sound and complete
 - Sound: If there is a *Simplex* derivation to SAT (or UNSAT), the formula is indeed satisfiable (or not).
 - Complete: For every formula, there is a *Simplex* derivation to SAT or UNSAT
- Problem is in P but unknown whether *Simplex* is in P

Simplex example



- No activation functions
- Property being checked: for $x_1 \in [0, 1]$, always $x_4 \notin [0.5, 1]$
 - Negated output property: $x_1 \in [0, 1]$ and $x_4 \in [0.5, 1]$

Simplex example



- Equations for weighted sums:

$$x_2 - x_1 = 0$$

$$x_3 + x_1 = 0$$

$$x_4 - x_3 - x_2 = 0$$

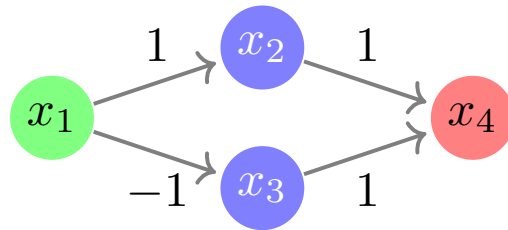
- Bounds:

$$x_1 \in [0, 1]$$

$$x_4 \in [0.5, 1]$$

x_2, x_3 unbounded

Simplex example



- Equations for weighted sums:

$$x_2 - x_1 = x_5$$

$$x_3 + x_1 = x_6$$

$$x_4 - x_3 - x_2 = x_7$$

- Bounds:

$$x_1 \in [0, 1]$$

$$x_4 \in [0.5, 1]$$

$$x_2, x_3 \text{ unbounded}$$

$$x_5, x_6, x_7 \in [0, 0]$$

- Technicality: replace constants by *auxiliary* variables

Simplex example

$$x_5 = x_2 - x_1$$

$$x_6 = x_3 + x_1$$

$$x_7 = x_4 - x_3 - x_2$$

Lower B.	Var	Value	Upper B.
0	x_1	0	1
	x_2	0	
	x_3	0	
0.5	x_4	0	1
0	x_5	0	0
0	x_6	0	0
0	x_7	0	0

Simplex example

$$x_5 = x_2 - x_1$$

$$x_6 = x_3 + x_1$$

$$x_7 = x_4 - x_3 - x_2$$

Update:

$$x_4 := x_4 + 0.5$$

Lower B.	Var	Value	Upper B.
0	x_1	0	1
	x_2	0	
	x_3	0	
0.5	x_4	0.5	1
0	x_5	0	0
0	x_6	0	0
0	x_7	0.5	0

Simplex example

$$x_5 = x_2 - x_1 \quad \leftarrow \quad x_5 = x_4 - x_3 - x_7 - x_1$$

$$x_6 = x_3 + x_1$$

$$x_7 = x_4 - x_3 - x_2 \quad \leftarrow \quad x_2 = x_4 - x_3 - x_7$$

Pivot: x_7, x_2

Lower B.	Var	Value	Upper B.
0	x_1	0	1
	x_2	0	
	x_3	0	
0.5	x_4	0.5	1
0	x_5	0	0
0	x_6	0	0
0	x_7	0.5	0

Simplex example

$$x_5 = x_4 - x_3 - x_7 - x_1$$

$$x_6 = x_3 + x_1$$

$$x_2 = x_4 - x_3 - x_7$$

Update:

$$x_7 := x_7 - 0.5$$

Lower B.	Var	Value	Upper B.
0	x_1	0	1
	x_2	0.5	
	x_3	0	
0.5	x_4	0.5	1
0	x_5	0.5	0
0	x_6	0	0
0	x_7	0	0

Simplex example

$$x_5 = x_4 - x_3 - x_7 - x_1 \quad \leftarrow \quad x_1 = x_4 - x_3 - x_7 - x_5$$

$$x_6 = x_3 + x_1 \quad \leftarrow \quad x_6 = x_4 - x_7 - x_5$$

$$x_2 = x_4 - x_3 - x_7$$

Pivot: x_5, x_1

Lower B.	Var	Value	Upper B.
0	x_1	0	1
	x_2	0.5	
	x_3	0	
0.5	x_4	0.5	1
0	x_5	0.5	0
0	x_6	0	0
0	x_7	0	0

Simplex example

$$x_1 = x_4 - x_3 - x_7 - x_5$$

$$x_6 = x_4 - x_7 - x_5$$

$$x_2 = x_4 - x_3 - x_7$$

Update:

$$x_5 := x_5 - 0.5$$

Lower B.	Var	Value	Upper B.
0	x_1	0.5	1
	x_2	0.5	
	x_3	0	
0.5	x_4	0.5	1
0	x_5	0	0
0	x_6	0.5	0
0	x_7	0	0

Simplex example

$$x_1 = x_4 - x_3 - x_7 - x_5$$

$$x_6 = x_4 - x_7 - x_5$$

$$x_2 = x_4 - x_3 - x_7$$

Failure

Lower B.	Var	Value	Upper B.
0	x_1	0.5	1
	x_2	0.5	
	x_3	0	
0.5	x_4	0.5	1
0	x_5	0	0
0	x_6	0.5	0
0	x_7	0	0

Simplex to Reluplex

- Solves satisfiability of conjunction of linear inequalities **and ReLU equations**.

$$\varphi = v_1 + v_2 \leq -5 \wedge v_1 - v_2 \geq 3 \wedge v_1 = \text{ReLU}(v_2)$$

- Step 1: Construct an initial **configuration** $C = (V, B, T, u, l, \alpha, R)$

$$V = \{v_1, v_2, v_3, v_4\}$$

- $B = \{v_3, v_4\}$
- $T : v_3 = v_1 + v_2$ and $v_4 = v_1 - v_2$
- $u(v_3) = -5$ and $l(v_4) = 3$
- $\alpha(v_i) = 0$ for all $v_i \in V$
- $R = \{(v_1 = \text{ReLU}(v_2))\}$

Set of ReLU constraints R

Simplex to Reluplex

Naïve approach:

$$\psi = v_1 + v_2 \leq 5 \wedge v_1 - v_2 \geq 3 \wedge v_1 = \text{ReLU}(v_2)$$



$$\psi = v_1 + v_2 \leq 5 \wedge v_1 - v_2 \geq 3 \wedge v_2 \leq 0 \wedge v_1 = 0$$

$\vee$

$$v_1 + v_2 \leq 5 \wedge v_1 - v_2 \geq 3 \wedge v_2 > 0 \wedge v_1 = v_2$$

Simplex to Reluplex

- Each ReLU node represented as two variables with a ReLU constraint
- The ReLU variables change independently
 - May violate ReLU constraint
 - Fix incrementally, similar to bound constraints
- Use pivots and updates, same as before

Modified successful termination test

$$\frac{\forall v_i \in V. l(v_i) \leq \alpha(v_i) \leq u(v_i), \quad \forall (v_f = \text{ReLU}(v_b)) \in R. \alpha(v_f) = \text{ReLU}(\alpha(v_b))}{\text{SAT}} \quad \text{ReluSuccess}$$

Current valuation meets all lower/upper bounds and all ReLU constraints

Additional pivot rule

$$\frac{v_i \in B, \quad \exists v_j. (v_j = \text{ReLU}(v_i)) \in R \vee (v_i = \text{ReLU}(v_j)) \in R, \quad v_e \notin B, T_{i,e} \neq 0}{T := \text{pivot}(T, v_i, v_e), \quad B := B \cup \{v_e\} \setminus \{v_i\}} \text{ReluPivot}$$

If a basic variable v_i is involved in a ReLU constraint,
Then swap it's a role with a non-basic variable v_e in its RHS with non-zero coefficient using pivot

Additional update rules

$$\frac{v_j \notin B, \quad (v_j = \text{ReLU}(v_i)) \in R, \quad \alpha(v_j) \neq \text{ReLU}(\alpha(v_i))}{\alpha := \text{update}(\alpha, v_j, \text{ReLU}(\alpha(v_i)) - \alpha(v_j))} \quad \text{Update}_f$$
$$\frac{v_i \notin B, \quad (v_j = \text{ReLU}(v_i)) \in R, \quad \alpha(v_j) \neq \text{ReLU}(\alpha(v_i)), \quad \alpha(v_j) \geq 0}{\alpha := \text{update}(\alpha, v_i, \alpha(v_j) - \alpha(v_i))} \quad \text{Update}_b$$

If a non-basic variable v_i is involved in a ReLU constraint that's violated by its current value,
Then update its value to satisfy the ReLU constraint

New split rule

$$\frac{(v_j = \text{ReLU}(v_i)) \in R, \quad l(v_i) < 0, \quad u(v_i) > 0}{u(v_i) := 0 \quad l(v_i) := 0} \quad \text{ReluSplit}$$

For the constraint $v_j = \text{ReLU}(v_i) = \max(0, v_i)$, when v_i can be both positive and negative we have two cases:

Case 1: v_i is positive (achieved by setting its lower bound to 0)

Case 2: v_i is negative (achieved by setting its upper bound to 0)

The two cases create two branches in the derivation tree

A priori we don't know which one will lead to success (so may require backtracking in proof search)

Case split is due to non-linearity of ReLU and crux of computational difficulty

Reluplex algorithm in SMT solver

- Starting with initial configuration, keep applying rules until SAT leaf found or all branches caused by split lead to UNSAT
- Multiple rules may be applicable in a given configuration
 - Derivation tree captures all possible branches of rule applications
 - Key difference with *Simplex*: Backtracking (exploring different branches) may be required!
 - Key benefit of Reluplex: Case split is demand driven and happens only when necessary
- Key engineering details of implementation
 - Which rule to choose in a given configuration
 - Choice of rules affects backtracking and time to termination
 - How to apply rules efficiently and how to backtrack efficiently

Properties of Reluplex

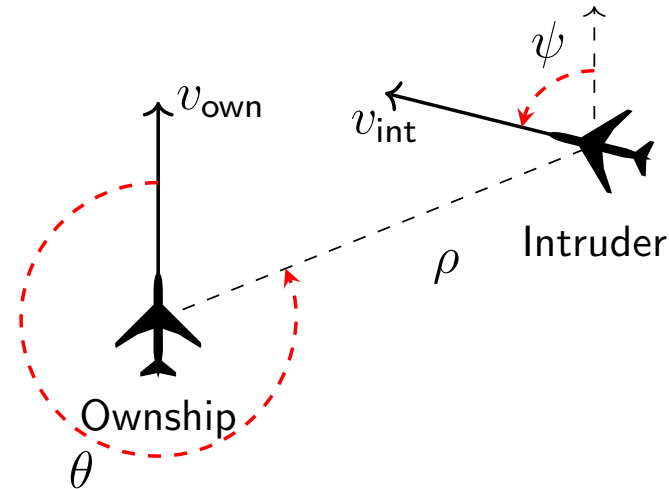
- Reluplex is sound and complete
 - Sound
 - If there is a Reluplex derivation tree with at least one SAT leaf, the formula is indeed satisfiable
 - If there is a Reluplex derivation tree with all UNSAT leaves, the formula is not satisfiable.
 - Complete: For every formula, there is a Reluplex derivation tree with a SAT or all UNSAT leaves
- Problem is NP complete
- Naive approach: split on all ReLU variables immediately
 - Exhaustive case-splitting approach from before
 - Ensures termination

More efficient Reluplex

- Better approach: lazy splitting
 - Start fixing bound violations
 - Once all variables within bounds, address broken ReLUs
 - If a ReLU is repeatedly broken, split on it
 - Otherwise, fix it without splitting
 - And repeat as needed
- Usually end up splitting on a fraction of the ReLUs (20%)
- Can reduce splitting further with some additional work

Reluplex evaluation: Acas Xu System

- An *Airborne Collision-Avoidance System*, for drones
- Being developed by the US Federal Aviation Administration (FAA)
- Produce an advisory:
 - *Clear-of-conflict (COC)*
 - *Strong left*
 - *Weak left*
 - *Strong right*
 - *Weak right*
- Implemented using neural networks



Certifying Acas Xu

- There are properties that the FAA cares about
 - Consistent alerting regions
 - No unnecessary turning advisories
 - Strong alerts do not occur when intruder vertically distant
- Properties defined formally
 - Constraints on inputs and outputs
- Example 1:
 - If the intruder is near and approaching from the left, the network advises strong right
 - Distance: $12000 \leq \rho \leq 62000$
 - Angle to intruder: $0.2 \leq \theta \leq 0.4$
 - Etc.

Comparison with existing SMT solvers

- Can encode $v_1 = \text{ReLU}(v_2)$ as $(v_2 \geq 0 \wedge v_1 = v_2) \vee (v_2 \leq 0 \wedge v_1 = 0)$.
- Existing SMT solvers perform many case splits.
- Reluplex can avoid/reduce splitting by using new pivot and update rules first.

	φ_1	φ_2	φ_3	φ_4	φ_5	φ_6	φ_7	φ_8
CVC4	-	-	-	-	-	-	-	-
Z3	-	-	-	-	-	-	-	-
Yices	1	37	-	-	-	-	-	-
MathSat	2040	9780	-	-	-	-	-	-
Gurobi	1	1	1	-	-	-	-	-
Reluplex	8	2	7	7	93	4	7	9

Time to termination in seconds with 4 hour timeout

Properties are from case study of neural-network-based controller for collision avoidance protocol ACAS (see ReluPlex paper; Katz et al; CAV 2017)

Experiments

Table Local adversarial robustness tests. All times are in seconds.

	$\delta = 0.1$		$\delta = 0.075$		$\delta = 0.05$		$\delta = 0.025$		$\delta = 0.01$		Total	
	Result	Time	Result	Time	Result	Time	Result	Time	Result	Time	Time	
Point 1	SAT	135	SAT	239	SAT	24	UNSAT	609	UNSAT	57	1064	SAT : Not Robust
Point 2	UNSAT	5880	UNSAT	1167	UNSAT	285	UNSAT	57	UNSAT	5	7394	UNSAT : Robust
Point 3	UNSAT	863	UNSAT	436	UNSAT	99	UNSAT	53	UNSAT	1	1452	
Point 4	SAT	2	SAT	977	SAT	1168	UNSAT	656	UNSAT	7	2810	
Point 5	UNSAT	14560	UNSAT	4344	UNSAT	1331	UNSAT	221	UNSAT	6	20462	

Neural Network: Fully connected, 8 layers, 300 neurons